EDUCATIONAL USE ONLY

CYBERSECURITY GUIDE

ENHANCED EDITION V3.0 — 14 CHAPTERS • LAB INCLUDED

A Student's Guide to Evilginx2

Master the art of understanding man-in-the-middle phishing attacks, session hijacking, and modern defense strategies.

> Comprehensive coverage of AiTM attack mechanics, phishlet configurations, session cookie theft, 2FA bypass techniques, attacker playbooks, real-life attack simulations, countermeasure analysis, and an intelligent hands-on practice lab. For authorized educational use only.

Cysec Don

cysecdon@gmail.com

2026

Table of Contents

DISCLAIMER: This guide is intended strictly for educational purposes. The techniques described herein should only be used in authorized penetration testing scenarios with explicit written consent from the target organization. Unauthorized use of these techniques is illegal and may result in criminal prosecution, civil liability, and severe penalties including imprisonment. The authors and publishers of this guide condemn any malicious use of cybersecurity tools and accept no responsibility for misuse.

Written by Cysec Don | cysecdon@gmail.com

Chapter 1: What in the World is Evilginx2?

Imagine you are writing a heartfelt letter to your bank, asking them to increase your credit limit. You hand the letter to a friendly postman who promises to deliver it. But this postman is not who he says he is. He reads your letter, copies your signature, delivers the letter to the bank on your behalf, and then hands you the bank's response as if nothing happened. Meanwhile, he now has a perfect copy of your signature and can write letters to your bank anytime he wants. That, in a nutshell, is the kind of attack Evilginx2 enables, and it is far more dangerous than simply stealing a password.

Evilginx2 is a man-in-the-middle (MITM) attack framework specifically designed to bypass two-factor authentication (2FA). Created by a Polish security researcher named Kuba Gretzcky in 2017, it evolved from the original evilginx project, which was more limited in scope and capability. The original tool was primarily a proof-of-concept, but Evilginx2 was built to be a fully functional phishing toolkit that could defeat the very security measures that most people trust to keep their accounts safe. It operates as a reverse proxy, sitting invisibly between the victim and the legitimate website, capturing everything in transit.

Why should students learn about Evilginx2? The answer is simple: you cannot defend against what you do not understand. Cybersecurity is not just about building walls; it is about understanding how those walls can be circumvented. By studying tools like Evilginx2, you gain insight into the mind of an attacker, which allows you to think critically about the security systems you interact with every day. Whether you aspire to be a penetration tester, a security analyst, or simply a more informed user, understanding these attack vectors is essential knowledge in the modern digital landscape.

It is critically important to emphasize that this guide is for educational purposes only. Using Evilginx2 or similar tools against any system without explicit, written authorization is a violation of computer crime laws in virtually every jurisdiction on Earth. The knowledge shared here should be used to strengthen defenses, not to cause harm. Think of it like studying lock-picking: a locksmith learns the craft to help people who are locked out, not to break into houses. The same principle applies here.

Chapter 2: The Magic Behind the Curtain - How Evilginx2 Works

To understand Evilginx2, you first need to understand the concept of a reverse proxy. Imagine a spy who puts on a postman's uniform and stands between you and the post office. When you give a letter to the "postman," he reads it, copies anything interesting, and then delivers it to the real post office. When the post office sends a reply, he intercepts it, reads that too, and then hands it to you. Neither you nor the post office ever realize there is an intermediary. A reverse proxy works in exactly the same way in the digital world: it sits between the user and the real web server, forwarding requests and responses while silently observing and capturing everything that passes through.

A Man-in-the-Middle (MITM) attack is a broad category of attack where an adversary positions themselves between two parties who believe they are communicating directly. In the context of Evilginx2, the two parties are the victim (you) and the legitimate website (your bank, email provider, or social media platform). The attacker becomes the invisible middleman, able to read, modify, or capture any data flowing between the two endpoints. This is fundamentally different from traditional phishing, where an attacker simply creates a fake copy of a website and hopes the victim does not notice the differences.

The key distinction between traditional phishing and Adversary-in-the-Middle (AiTM) phishing is that traditional phishing is static while AiTM is dynamic. In a traditional phishing attack, the fake site is a snapshot, a frozen copy of the real site. It cannot respond to changes on the real server, and it certainly cannot process your two-factor authentication code in real time. An AiTM attack, by contrast, proxies everything in real time. The victim interacts with what appears to be the genuine website because, in a sense, it IS the genuine website, just viewed through a malicious lens. The proxy fetches live content from the real server, so the victim sees the actual site with all its dynamic features, current content, and valid security indicators.

The proxy sits between the victim and the real website, relaying traffic in both directions. When the victim types in their username and password, the proxy captures those credentials and forwards them to the real site. When the real site asks for a 2FA code, the proxy presents that prompt to the victim, who dutifully enters the code. The proxy forwards the code to the real site, and the real site grants access. But here is the critical part: the proxy also captures the session token, the digital key that proves you are logged in.

Session tokens versus passwords is a crucial distinction, and here is the best analogy: think of a password as the key to your hotel room, and a session token as the electronic keycard that the front desk gives you after you check in. If someone steals your key (password), you can change the lock. But if someone copies your keycard (session token), the hotel has no idea that a duplicate exists. The attacker can walk right into your room, and the hotel thinks it is you. This is why session token theft is so

devastating: the real server has no way to distinguish between the legitimate user and the attacker who holds the stolen token.

Chapter 3: Setting Up Your Lab (Without Getting Arrested)

Before you even think about downloading Evilginx2, let us make one thing absolutely clear: using this tool against any system without explicit written authorization is illegal. Period. Full stop. No exceptions. The Computer Fraud and Abuse Act (CFAA) in the United States, the Computer Misuse Act in the United Kingdom, and similar laws in virtually every country treat unauthorized access to computer systems as a serious crime punishable by years in prison and massive fines. Using this on your roommate's Gmail is NOT a lab exercise, no matter how curious you are about whether they would fall for it. That is a federal crime, not a science experiment.

With that out of the way, let us talk about setting up a legitimate lab environment. You will need a few prerequisites. First, you need a VPS (Virtual Private Server) running Linux. Ubuntu 20.04 or 22.04 LTS works well. You will also need a domain name that you control. This domain should NOT resemble any real organization's domain. Using something like "totally-not-a-scam.example.com" is fine for testing. The Go programming language is also required, as Evilginx2 is written in Go. You can install it with the following commands:

```
sudo apt update
sudo apt install -y golang git
git clone https://github.com/kgretzky/evilginx2.git
cd evilginx2
make
sudo ./evilginx2
```

DNS configuration is a critical step. Evilginx2 needs to act as the authoritative DNS server for your test domain. You will need to configure your domain registrar to point the NS records for your test domain to your VPS IP address. This allows Evilginx2 to issue valid TLS certificates for your phishing domain via Let's Encrypt, making the phishing site appear legitimate with a valid HTTPS padlock in the browser. Remember, this should only be done with domains you own, for testing purposes only.

Setting up a safe testing environment means using only your own accounts, your own domains, and your own infrastructure. Create test accounts on services you control, and use those for all your experiments. Document everything you do, keep logs, and ensure that no real users are ever exposed to your test infrastructure. If you are doing this as part of a university course or professional engagement, ensure you have written authorization from the relevant stakeholders before proceeding. The paperwork is not just a formality; it is your legal protection.

Chapter 4: Phishlets - The Recipe for Trouble

Phishlets are the heart and soul of Evilginx2. Think of them as the recipe cards that tell Evilginx2 how to proxy a specific website. If Evilginx2 is a fake storefront, then phishlets are the blueprints that make the fake Starbucks look like the real one right down to the font on the menu board, except the barista is quietly copying your credit card information while making your latte. Each phishlet is a YAML configuration file that defines which website to proxy, which subdomains to intercept, which cookies to capture, and how to modify the traffic flowing through the proxy.

A phishlet tells Evilginx2 three essential things: first, which legitimate website should be proxied (for example, accounts.google.com); second, which subdomains of the phishing domain should be used (for example, login.your-test-domain.com); and third, which session cookies should be captured from the victim's browser after a successful login. Without a phishlet, Evilginx2 does not know what to do. It is like giving someone a flashlight without telling them what they are looking for.

Evilginx2 comes with several built-in phishlets for popular services such as Google, Microsoft, LinkedIn, Twitter, and others. These built-in phishlets are written in YAML format and follow a specific structure. Let us look at the key components of a phishlet file. The hostname field specifies the primary domain of the target service. The subfilters section defines how URLs in the proxied content should be rewritten to point to the phishing domain. The auth_urls section lists the URLs where authentication occurs, so Evilginx2 knows when to start and stop capturing. Finally, the cookies section specifies which session cookies should be extracted after authentication is complete.

```
# Example phishlet structure (simplified)
name: example-service
hostname: login.example.com
subfilter:
- hostname: accounts.example.com
- hostname: www.example.com
auth_urls:
- /login
- /authenticate
cookies:
- sid
- session_token
- auth_id
```

Customizing phishlet configurations requires a solid understanding of how the target website handles authentication. You need to know which URLs are involved in the login flow, which cookies are set after successful authentication, and which subdomains are used for serving static content. This information can be gathered by observing the network traffic during a normal login using browser developer tools. Remember, custom phishlets should only be created and tested against services and accounts you own or have explicit authorization to test.

Chapter 5: Running Your First Simulation

Now that you have your lab set up and understand phishlets, let us walk through launching your first simulation campaign. This section describes a fictional scenario we will call the "Free Pizza" phish, because honestly, who would not click on a link promising free pizza? The psychology is simple: offer something irresistible, and people will click. That is the core of every successful phishing campaign, and understanding this psychology is the first step toward building better defenses.

The first step is to configure your phishlet for the target service. In Evilginx2's CLI, you would use commands like the ones shown in the table below. After loading the phishlet, you need to configure lures, which are the actual phishing links that will be sent to the target. A lure consists of a redirect URL (where the victim goes after the attack), the phishing URL (the link the victim clicks), and optional parameters for tracking. The lure system in Evilginx2 is quite sophisticated, allowing you to generate unique URLs for each target and track which ones were clicked.

Monitoring captured sessions is done through the Evilginx2 CLI as well. When a victim successfully authenticates through the proxy, Evilginx2 captures the session cookies and stores them. You can view captured sessions using the sessions command. Each captured session includes the victim's username, the captured cookies (including session tokens), the timestamp of the capture, and the remote IP address of the victim. The data captured typically includes authentication cookies, session tokens, and sometimes the username and password that were entered during the login process.

Table 1: Key Evilginx2 CLI Commands

Command	Description	Example
config	View or set configuration	config domain test.com
phishlets	List available phishlets	phishlets
phishlet enable	Enable a specific phishlet	phishlet enable google
phishlet hostname	Set phishing hostname	phishlet hostname google phish.test.com
lures create	Create a new phishing lure	lures create google
lures get-url	Get the phishing URL	lures get-url 0
sessions	List captured sessions	sessions
sessions delete	Delete a captured session	sessions delete 0

Chapter 6: The Cookie Monster - Session Hijacking Explained

Session cookies are the unsung heroes of the modern web. When you log into a website, the server creates a session for you and sends a cookie to your browser as a kind of VIP wristband. Think of it like getting a wristband at a club: you show your ID at the door (enter your password and 2FA), the bouncer gives you a wristband (the session cookie), and from that point on, you just flash the wristband to get back in without showing your ID again. This is convenient, but it also means that anyone who gets hold of your wristband can walk right into the club, and the bouncer will think it is you.

Why is stealing a cookie worse than stealing a password? Because changing your password does not invalidate the session cookie. If an attacker steals your password, you can change it, and the attacker is locked out. But if an attacker steals your session cookie, they have access to your account right now, and changing your password does nothing to stop them. The session cookie is independent of your password. It is like changing the lock on your front door while the thief is already sitting on your couch, holding a spare key that still works on the new lock. The only way to kick them out is to explicitly revoke the session, which many users do not know how to do, and which many services make surprisingly difficult.

Session persistence and token lifetimes vary significantly between services. Some services issue session tokens that expire after 30 minutes of inactivity, while others keep sessions alive for weeks or even months. The "remember me" checkbox on login forms is perhaps the ultimate trap in this context. When you check that box, you are asking the server to issue a long-lived session token that may not expire for 30, 60, or even 90 days. If an attacker captures one of these persistent tokens, they effectively have permanent access to your account for as long as the token remains valid.

Different services handle sessions differently, and this affects the severity of a session token theft. Google, for example, uses a system where session tokens can be refreshed and may remain valid for extended periods. Some banking applications use short-lived tokens that expire within minutes, making stolen tokens less useful. Social media platforms tend to use very long-lived tokens for convenience. Understanding these differences is crucial for both attackers and defenders, as it determines the window of opportunity after a token is stolen.

Table 2: Password Theft vs. Cookie/Token Theft

Aspect	Password Theft	Cookie / Token Theft
How it works	Attacker obtains your password	Attacker obtains your session cookie
2FA protection	2FA blocks unauthorized login	2FA is already bypassed; cookie proves authentication
Remediation	Change password to revoke access	Must explicitly revoke the session; changing password is insufficient
Detection	Unusual login notifications possible	Attacker appears as legitimate user; very hard to detect
Persistence	Password change = attacker locked out	Token may persist for days/weeks after password change
Severity	High	Critical

Chapter 7: Bypassing 2FA - The Plot Twist

Two-factor authentication is supposed to be the bouncer at the door of your digital life. You show your ID (password), and then the bouncer asks for a second form of verification, like checking your phone for a code or confirming a push notification. It adds an extra layer of security that has saved countless accounts from being compromised. But what if the bouncer has a blind spot? What if there is a way to slip past while he is checking someone else's ID? That is exactly what Evilginx2 exploits: the bouncer checks the ID properly, but the attacker is standing right next to you, listening to every word you say to the bouncer.

The reason Evilginx2 bypasses 2FA is elegantly simple: it does not attack the 2FA mechanism at all. Instead, it proxies the entire authentication flow in real time. When the real website asks the victim for a 2FA code, the proxy presents that exact same prompt to the victim. The victim enters their code, the proxy captures it and immediately forwards it to the real website, and the real website validates it. The 2FA system works exactly as intended; the code is correct, the verification succeeds, and the session is established. But the proxy has been watching the entire exchange, and it captures the session token that the real website issues after successful authentication.

SMS-based 2FA is particularly vulnerable to this type of attack because of how it is implemented. When a website sends a one-time code to your phone, that code is valid for a certain time window, usually 30 seconds to 10 minutes. The proxy simply relays the prompt to the victim, who enters the code they received via SMS. The proxy forwards it to the real site within seconds. The time-based nature of SMS codes provides no protection against a real-time proxy attack because the proxy operates

instantaneously.

Push-based 2FA, where you receive a notification on your phone asking you to approve or deny the login, is also vulnerable. When the real site sends a push notification to the victim's phone, the victim sees a legitimate-looking login attempt and approves it, not realizing that the request is actually being funneled through a malicious proxy. The attacker gets the authenticated session token, and the victim has unknowingly authorized the attacker's access. This is why many security professionals now consider traditional 2FA methods insufficient against sophisticated phishing attacks.

The one defense that stands against this attack is FIDO2/WebAuthn, which we can think of as the "biometric bouncer." FIDO2 uses public-key cryptography and binds the authentication to the specific origin (domain) of the website. When a FIDO2 security key authenticates you, it signs a challenge that includes the website's domain. If the domain does not match, the authentication fails. Since Evilginx2 must use a different domain for the phishing site, the FIDO2 key will refuse to authenticate, effectively blocking the attack. This is why FIDO2 is considered the gold standard for phishing-resistant authentication.

Chapter 8: Defense - How to Not Get Evilginx'd

Defending against Evilginx2 and similar AiTM attacks requires a multi-layered approach. The first line of defense is awareness. Learning to detect phishing proxies starts with carefully checking URLs. Yes, the phishing site will have a valid HTTPS certificate, but the domain name will be different from the real one. If your bank's URL is "totally-not-a-scam.example.com," maybe do not log in there. The domain is the single most important indicator of a proxy attack, but it requires users to actually pay attention to what is in their address bar, which surprisingly few people do.

FIDO2/WebAuthn is the gold standard defense against AiTM phishing attacks. As discussed in the previous chapter, FIDO2 binds authentication to the specific domain, making it impossible for a proxy on a different domain to replay the authentication. Organizations should prioritize deploying FIDO2 security keys or platform authenticators for all high-value accounts. The initial cost of security keys is negligible compared to the cost of a successful phishing attack, which can run into millions of dollars in damages, lost data, and reputational harm.

Security awareness training is another critical component of defense. Users need to understand that a valid HTTPS certificate and a professional-looking website are not guarantees of legitimacy. Training should focus on teaching users to verify domain names, be suspicious of unsolicited links, and report anything that seems even slightly off. The best training programs use simulated phishing exercises to give users hands-on experience in identifying phishing attempts, including AiTM attacks where the fake site looks completely legitimate.

Modern browsers include several security features that can help detect or prevent AiTM attacks. Browser extensions that check domain reputation, password managers that only autofill on the correct domain, and built-in phishing detection all add layers of protection. Corporate defense strategies should include conditional access policies that require device trust, compliance checks, and risk-based authentication. If a login attempt comes from an unrecognized device or an unusual location, conditional access can require additional verification that a proxy cannot provide, such as a device-specific certificate.

Table 3: Defensive Measures and Their Effectiveness

Defensive Measure	Effectiveness	Notes
FIDO2/WebAuthn	Very High	Domain-bound; defeats AiTM proxy attacks entirely
Conditional Access / Device Trust	High	Requires managed devices; limits proxy utility
Security Awareness Training	Medium-High	Users learn to verify domains; human factor still variable
Password Managers	Medium	Only autofill on correct domain; prevents credential entry on phishing sites
Browser Phishing Detection	Medium	Automated checks; may miss sophisticated proxy attacks
SMS / Push-based 2FA	Low against AiTM	Easily bypassed by real-time proxy
Checking URLs Manually	Medium	Effective if done consistently; often neglected in practice

Chapter 9: Legal and Ethical Considerations

The legal landscape surrounding cybersecurity tools like Evilginx2 is serious and unforgiving. In the United States, the Computer Fraud and Abuse Act (CFAA) makes it a federal crime to access a computer system without authorization or to exceed authorized access. Violations can result in up to 10 years in prison for a first offense and up to 20 years for repeat offenses. The UK's Computer Misuse Act carries similar penalties, and the EU's directive on attacks against information systems ensures that similar laws exist across all member states. Simply put, there is no jurisdiction where using Evilginx2 against unauthorized targets is legal.

Ethical hacking guidelines exist to provide a framework for legitimate security testing. The key principles are: always obtain written authorization before testing, define the scope of testing in advance,

do not access or modify data beyond what is necessary for the test, report all findings to the client, and destroy any captured data after the engagement is complete. Professional penetration testers follow these rules religiously, not just because it is ethical, but because it is the law. The rules of engagement are typically documented in a Statement of Work (SOW) or a Rules of Engagement (ROE) document, both of which must be signed by the client before any testing begins.

When is it OK to use Evilginx2? The answer is straightforward: only during authorized penetration tests with written consent from the target organization. This means you need a signed engagement letter, a clear scope document, and explicit permission to use phishing techniques as part of the test. Many penetration testing engagements explicitly exclude phishing, so even having a general authorization to test may not cover the use of tools like Evilginx2. Always clarify the scope and get specific authorization for the techniques you plan to use.

The consequences of misuse cannot be overstated. Beyond criminal prosecution and potential prison time, individuals caught conducting unauthorized phishing attacks face civil lawsuits, career destruction, and permanent damage to their professional reputation. Many cybersecurity professionals have seen their careers ended by a single poor decision made in a moment of curiosity or ambition. Responsible disclosure practices, where security researchers report vulnerabilities to the affected organizations rather than exploiting them, are the ethical standard in the industry. Getting proper authorization is not just a legal requirement; it is the foundation of a career in cybersecurity.

Chapter 10: The Attacker's Playbook - Common Tricks and Techniques

Understanding how real attackers operate is essential for building effective defenses. This chapter examines the most common techniques used in AiTM phishing campaigns, providing detailed technical explanations and actual command examples. Every technique described here should only be used in authorized testing environments. The purpose of revealing these methods is to help defenders understand what they are up against and to design countermeasures accordingly.

Domain Squatting and Typo-Squatting

Attackers frequently register domains that closely resemble legitimate ones, relying on the fact that most users do not carefully examine URLs. Domain squatting involves registering variations of popular domains that exploit common typos or visual similarities between characters. For example, an attacker might register "g00gle.com" (using zeros instead of the letter "o"), "micr0soft-login.com", or "secure-0nline.com". These lookalike domains are combined with Evilginx2 to create convincing phishing pages. The attacker configures the phishlet hostname to use a subdomain of the squatted domain, such as "login.g00gle.com", making the URL appear even more legitimate at a glance.

URL Shortening and Redirect Chains

To hide the true destination of a phishing link, attackers commonly use URL shortening services such as bit.ly, TinyURL, or custom redirect scripts. When a victim sees a shortened URL like "bit.ly/3xHj9kM", they have no way of knowing where it leads without clicking it. More sophisticated attackers set up redirect chains: the victim clicks a shortened link that leads to a compromised legitimate site, which then redirects to another URL, and finally arrives at the Evilginx2 proxy. This multi-hop approach evades URL scanners and security gateways that follow links to check their destination.

Email Spoofing and Social Engineering Pretexts

Crafting a convincing phishing email is an art that combines technical skill with social engineering psychology. Attackers use email spoofing techniques to bypass SPF, DKIM, and DMARC protections. This often involves finding misconfigured email servers or using services that allow custom sender addresses. The social engineering pretext is the story the email tells to convince the victim to click. Common pretexts include: IT department notifications about password expiry, HR announcements about policy changes, CEO fraud (impersonating an executive), package delivery notifications, and security alerts claiming suspicious activity on the victim's account.

Customizing Lures in Evilginx2

Evilginx2 provides a powerful lure system that allows attackers to customize every aspect of the phishing experience. After creating a lure with the "lures create" command, attackers use "lures edit" to configure the redirect URL, the path, and other parameters. For example, the command "lures edit 0 redirect_url https://real-site.com" sets the page the victim sees after completing the attack, making it appear as though nothing unusual happened.

```
config domain yourtest.com
phishlet enable o365
phishlet hostname o365 login.yourtest.com
lures create o365
lures edit 0 redirect_url https://office.com
lures edit 0 path /security-update
lures get-url 0
```

Blacklisting and Evasion

Experienced attackers use Evilginx2's blacklisting feature to prevent security researchers and automated scanners from discovering their phishing pages. The "blacklist" command blocks specific IP addresses or CIDR ranges from accessing the proxy. Attackers maintain lists of known security vendor IP ranges, URL scanner IP addresses, and threat intelligence company netblocks, adding them to the blacklist before launching a campaign.

Campaign Management and Session Export

Evilginx2 includes a campaign management system that allows attackers to run multiple phishing campaigns simultaneously, each targeting different services or organizations. The "campaign" command provides functionality for creating, managing, and tracking distinct campaigns with their own lures, phishlets, and captured sessions. After a successful capture, attackers use the "sessions" command to view stolen credentials and cookies. The data can be exported in JSON format for programmatic session replay.

Chapter 11: Real-Life Simulation - The "IT Helpdesk" Attack

This chapter presents a detailed, step-by-step simulation of a realistic AiTM phishing attack against a fictional mid-size company called "Acme Corp." The attacker, whom we will call "Marcus," targets Acme Corp's Office 365 environment. This simulation is entirely fictional and is presented for educational purposes only. Every step is analyzed from both the attacker's and defender's perspectives, with pitfalls highlighted at each stage.

Step 1: Reconnaissance

Marcus begins by gathering publicly available information about Acme Corp. Using LinkedIn, he identifies key employees, their job titles, and the email format the company uses. He discovers that Acme Corp uses the email format "firstname.lastname@acme.com" and that the company uses Office 365 for email and collaboration. He checks the company's website for press releases, blog posts, and employee directories. WHOIS lookups reveal that the domain acme.com was registered through a major registrar.

Step 2: Domain Registration

Marcus registers the domain "acme-c0rp-support.com", replacing the letter "o" in "corp" with the number zero. This is a classic typo-squatting technique that is difficult to spot at a glance, especially in email headers or URLs where the visual difference between "o" and "0" is minimal. He registers through a privacy-focused registrar using a prepaid card and a pseudonym.

> PITFALL: Domain reputation services and advanced threat protection systems may flag newly registered domains within hours of registration. Marcus's domain could be blacklisted by Safe Links or Microsoft Defender before any victims even receive his email.

Step 3: VPS Setup and Evilginx2 Installation

Marcus rents a VPS with the IP address 198.51.100.42 from a hosting provider that accepts cryptocurrency and does not require identity verification. He connects to the server and installs Evilginx2 with the following commands:

```
ssh root@198.51.100.42
apt update && apt upgrade -y
apt install -y golang-go git certbot
git clone https://github.com/kgretzky/evilginx2.git
cd evilginx2 && make
./build/evilginx2
```

PITFALL: Evilginx2's default certificate requests via Let's Encrypt have rate limits. If Marcus makes too many certificate requests during testing, he may be temporarily blocked from obtaining new certificates, delaying his campaign.

Step 4: DNS Configuration

Marcus configures his domain's DNS to point to the Evilginx2 server. At his domain registrar, he changes the NS records for acme-c0rp-support.com to point to his VPS IP. Then, inside the Evilginx2 console, he configures the domain, IP address, and phishlet:

```
config domain acme-c0rp-support.com
config ip 198.51.100.42
phishlet enable o365
phishlet hostname o365 login.acme-c0rp-support.com
```

PITFALL: DNS changes can take up to 48 hours to propagate globally, though they typically take only minutes to a few hours. During this window, some victims may not be able to reach the phishing page.

Step 5: Crafting the Phishing Email

Marcus crafts a convincing phishing email that appears to come from Acme Corp's IT Helpdesk. The email creates a sense of urgency by claiming that all employees must reset their passwords within 24 hours due to a security upgrade. The email includes the company logo, proper formatting, and convincing language designed to override the victim's natural skepticism with urgency and authority.

Step 6: Creating Lures

```
lures create o365
lures edit 0 redirect_url https://outlook.office.com
lures edit 0 path /it-password-reset
lures get-url 0
```

Step 7: The Victim Clicks

An Acme Corp employee named Jane Smith receives the email and clicks the link. Her browser connects to login.acme-c0rp-support.com, which resolves to Marcus's Evilginx2 server. Evilginx2 presents a valid HTTPS certificate (obtained via Let's Encrypt), so Jane sees the padlock icon in her

browser. The proxy fetches the real Microsoft login page in real time and presents it to Jane. She enters her username and password. Evilginx2 captures these credentials and simultaneously forwards them to the real Microsoft login page. Microsoft then prompts for a 2FA code. The proxy relays this prompt to Jane, who enters the code. The entire process takes only a few seconds longer than a normal login.

Step 8: Session Capture

```
[sessions]
ID | Time | Username | Remote IP | Capture
0 | 2026-05-15 09:23:41 | jsmith@acme.com | 203.0.113.55 | cookies + creds
```

PITFALL: If Jane uses FIDO2 authentication instead of an authenticator app, the entire attack fails. FIDO2 binds the authentication to the real domain (login.microsoftonline.com), not Marcus's phishing domain. The FIDO2 key simply refuses to authenticate on the phishing site.

Step 9: Session Replay

Marcus exports the captured session cookies using the "sessions" command and prepares to replay the session. He opens a fresh browser instance and installs the EditThisCookie browser extension. He navigates to outlook.office.com and injects the stolen session cookies into his browser. The browser sends these cookies to Microsoft's servers, which recognize them as valid and grant Marcus access to Jane's account without requiring a username, password, or 2FA code.

PITFALL: Conditional Access policies requiring compliant devices will block the replayed session. If Acme Corp requires that only Intune-managed devices can access Office 365, Marcus's unmanaged browser will be denied access regardless of having valid session cookies.

Step 10: Post-Compromise Access

Once inside Jane's account, Marcus can access her email inbox, OneDrive files, Teams messages, and SharePoint sites. He searches for sensitive documents, financial data, and credentials stored in emails. He may set up mailbox forwarding rules to maintain access even if the session expires, create new application passwords, or register his own device for multi-factor authentication to establish persistent access.

Chapter 12: Pitfalls, Countermeasures, and Staying One Step Ahead

Every attack technique has weaknesses, and every defensive measure has limitations. This chapter provides a comprehensive analysis of the pitfalls that both attackers and defenders face in the context of AiTM phishing attacks. Understanding these pitfalls from both perspectives is essential for building robust, phishing-resistant security architectures.

Table 4: Pitfalls, Attacker Perspective, and Defender Countermeasures

Pitfall	Attacker Perspective	Defender Countermeasure
Domain Detection	Newly registered domains are often flagged within hours by threat intel feeds and domain reputation services	Monitor certificate transparency logs for domains similar to yours; subscribe to domain squatting alert services
FIDO2/WebAuthn	FIDO2 completely defeats AiTM attacks by binding authentication to the origin domain; no workaround exists	Deploy FIDO2 security keys or passkeys for all users; make FIDO2 the primary MFA method
Conditional Access	CA policies requiring compliant or hybrid-joined devices block session replay from unmanaged machines	Enforce device trust and compliance checks; require Intune-managed devices for all cloud resource access
Session Expiration	Short-lived tokens may expire before the attacker can replay them	Configure short access token lifetimes; implement continuous session validation
IP Geolocation	Replaying sessions from a different country or unusual location triggers anomaly detection	Implement impossible-travel detection; require step-up authentication for logins from new locations
Device Trust	Sessions replayed from unknown or non-compliant devices are blocked by device trust policies	Require device registration and compliance verification; deploy certificate-based device authentication
Certificate Transparency	CT logs publicly expose all SSL certificates issued for phishing domains, enabling early detection	Monitor CT logs for certificates issued for typosquatting domains; set up automated alerts
URL Scanning Services	Automated URL scanners follow and analyze phishing URLs before victims click them	Ensure URL rewriting is enabled in email security; use link isolation technology
DNS Monitoring	Suspicious DNS configurations can be detected by DNS monitoring services	Monitor DNS records for unauthorized changes; use DNS security extensions (DNSSEC)
Browser Security Features	Modern browsers warn users about deceptive sites and block known phishing URLs	Enforce browser security policies organization-wide; deploy browser extensions that check domain reputation
Token Binding	Token binding cryptographically ties tokens to the TLS connection, making stolen tokens unusable	Enable token binding on all authentication flows; migrate to bound tokens where supported
DMARC/DKIM/SPF	Properly configured email authentication makes it difficult to spoof sender domains	Publish strict DMARC policies (p=reject); configure DKIM signing for all outgoing mail

Emerging Defenses

The security landscape is evolving rapidly, and several emerging technologies promise to significantly raise the bar for AiTM attackers. Token Binding is a proposed IETF standard that cryptographically binds security tokens to the TLS connection between the client and server. When token binding is in effect, a stolen session cookie cannot be replayed on a different TLS connection because the server verifies that the token's binding matches the current connection's cryptographic parameters.

Continuous Access Evaluation (CAE) is a Microsoft technology that enables near-real-time enforcement of access policies. Instead of relying solely on the initial authentication event, CAE continuously evaluates the user's session against current policies and risk signals. If a user's risk profile changes during an active session, CAE can revoke the session within minutes rather than waiting for the token to expire naturally.

Passkeys, the successor to traditional passwords based on the FIDO2 standard, represent the most promising path toward a phishing-resistant future. Unlike passwords, passkeys cannot be phished because the authentication is bound to the specific origin (domain) of the website. Even if a victim is tricked into visiting a phishing site, the passkey authenticator will refuse to generate a valid signature for the wrong domain.

Building a Phishing-Resistant Architecture

Organizations can build a phishing-resistant architecture by layering multiple defenses together. The foundation should be FIDO2 or passkey-based authentication for all users, supplemented by conditional access policies that enforce device trust and compliance checks. Continuous monitoring through CAE and behavioral analytics provides the ability to detect and respond to compromised sessions in near real time. Email security should include strict DMARC policies, URL rewriting and sandboxing, and advanced threat protection. DNS monitoring and certificate transparency log monitoring provide early warning of phishing infrastructure being set up. Regular security awareness training that specifically covers AiTM attacks and the importance of checking domain names ensures that human factors do not become the weakest link in the chain.

Chapter 13: Recap and Further Learning

Let us review the key takeaways from each chapter. In Chapter 1, we learned that Evilginx2 is an MITM attack framework created by Kuba Gretzcky in 2017, designed to bypass 2FA by acting as a malicious translator between victims and legitimate websites. Chapter 2 explained the reverse proxy concept, the difference between traditional phishing and AiTM attacks, and why session tokens are more valuable

than passwords.

Chapter 3 covered the prerequisites and steps for setting up a legal lab environment. Chapter 4 introduced phishlets. Chapter 5 walked through launching a simulation campaign. Chapter 6 explained session hijacking in detail. Chapter 7 revealed how Evilginx2 bypasses 2FA by proxying the authentication flow in real time, and identified FIDO2/WebAuthn as the only effective defense against this technique.

Chapter 8 provided a comprehensive defense guide. Chapter 9 discussed the serious legal consequences of misuse. Chapter 10 explored the attacker's playbook in detail. Chapter 11 presented a realistic step-by-step simulation. Chapter 12 provided a comprehensive table mapping every pitfall to its attacker implications and defender countermeasures, and discussed emerging defenses.

For further learning, we recommend the following resources: "The Web Application Hacker's Handbook" by Dafydd Stuttard and Marcus Pinto, "Penetration Testing" by Georgia Weidman, and the OWASP Testing Guide. Online platforms like Hack The Box, TryHackMe, and PortSwigger Web Security Academy provide hands-on practice environments. Certifications such as OSCP, CEH, and CISSP are valuable career milestones.

Career paths in cybersecurity are diverse and rewarding. Red team specialists focus on offensive security. Blue team defenders protect organizations by monitoring, detecting, and responding to threats. Purple team professionals combine both perspectives. Whichever path you choose, remember that ethical hacking is not just a skill set but a mindset: always seek to understand, always seek to protect, and always stay on the right side of the law.

Chapter 14: Simulated Practice Lab - Hands-On Exercises

There is a fundamental difference between reading about a tool and actually using it. The theoretical knowledge you have gained from the first thirteen chapters is essential, but it remains abstract until you put it into practice. This chapter provides a structured, safe, and progressive hands-on lab environment where you can transform theoretical understanding into practical skill. Our philosophy is simple: learning by doing, safely. Every exercise in this lab uses only infrastructure you own, accounts you control, and domains you have registered. No real users, no real organizations, no real harm. If you have not already secured a VPS, a test domain, and a test account, go back to Chapter 3 and complete that setup before proceeding.

The lab is structured as four progressive exercises. Each builds on the skills and infrastructure established in the previous one, mirroring the way a real penetration test unfolds: first you set up the environment, then you configure the attack surface, then you execute the attack, and finally you verify

the defenses. By the end of these four labs, you will have walked through the complete lifecycle of an AiTM phishing assessment from infrastructure provisioning to findings reporting. Take your time with each lab. Rushing through the steps without understanding them defeats the purpose of the exercise.

Table 5: Lab Environment Requirements

Component	Specification	Purpose
VPS	Ubuntu 22.04 LTS, 2 vCPU, 4GB RAM	Host for Evilginx2
Domain	Custom domain you own (NOT resembling real orgs)	DNS + TLS certificates
Test Account	Account on service you own/control	Safe target for simulation
Browser	Chrome/Firefox with DevTools	Analyze traffic and cookies
Cookie Editor	EditThisCookie or similar extension	Session replay testing
Password Manager	Bitwarden, 1Password, or equivalent	Test auto-fill behavior
FIDO2 Key	YubiKey or similar (optional)	Test phishing-resistant auth

Lab 1: Foundation - Setting Up a Safe Evilginx2 Environment

LEARNING OBJECTIVE: Students will be able to install, configure, and verify a working Evilginx2 instance in a safe, isolated environment. By completing this lab, you will understand the infrastructure requirements, network configuration, and security hardening steps necessary to run Evilginx2 for an authorized assessment.

SCENARIO: You are a junior pentester at CyberSec Corp. Your team lead has asked you to set up a test environment for an upcoming authorized phishing assessment. You need to get Evilginx2 running on a clean VPS with a test domain. The assessment is two weeks away, so you have time to build the environment correctly and document every step. Your team lead emphasized that the environment must be fully hardened before any phishlets are configured.

PREREQUISITES CHECKLIST:

Before starting this lab, verify you have the following: (1) A VPS running Ubuntu 22.04 LTS with at least 2 vCPU and 4GB RAM, (2) SSH root access to the VPS, (3) A domain name you own registered at a registrar where you can modify NS records, (4) The VPS public IP address noted and accessible, (5) A local machine with an SSH client and DNS lookup tools (dig or nslookup).

STEP-BY-STEP INSTRUCTIONS:

Step 1: Harden your VPS. Connect to your VPS via SSH and apply system updates, then configure the UFW firewall to allow only the ports that Evilginx2 requires. Port 22 for SSH management, port 53 for DNS (both TCP and UDP, since Evilginx2 acts as an authoritative DNS server), port 80 for HTTP (needed for ACME certificate challenges and HTTP-to-HTTPS redirects), and port 443 for HTTPS (the main phishing proxy traffic).

```
# Step 1: VPS hardening
ssh root@YOUR_VPS_IP
apt update && apt upgrade -y
ufw allow 22/tcp
ufw allow 53/tcp
ufw allow 53/udp
ufw allow 80/tcp
ufw allow 443/tcp
ufw enable
```

Step 2: Install Go and Evilginx2. Download and install the Go programming language, which is required to compile Evilginx2 from source. Then clone the Evilginx2 repository and build the binary. Note that building from source ensures you have the latest version and can audit the code if needed.

```
# Step 2: Install Go and Evilginx2
wget https://go.dev/dl/go1.21.0.linux-amd64.tar.gz
tar -C /usr/local -xzf go1.21.0.linux-amd64.tar.gz
export PATH=$PATH:/usr/local/go/bin
echo 'export PATH=$PATH:/usr/local/go/bin' >> ~/.bashrc
source ~/.bashrc
go version

git clone https://github.com/kgretzky/evilginx2.git
cd evilginx2 && make
```

Step 3: Configure Evilginx2. Launch Evilginx2 as root (required for binding to port 53 for DNS) and configure the base domain and IP address. Then verify that the phishlets list loads correctly and that your configuration is applied.

```
# Step 3: Configure Evilginx2
sudo ./evilginx2
config domain your-lab-domain.com
config ip YOUR_VPS_IP
phishlets
```

VERIFICATION STEPS:

To confirm your environment is correctly set up, perform the following checks. First, the phishlets command should display a list of available phishlets with their status. Second, the config command should show your configured domain and public IP address. Third, from your local machine, verify that your VPS is listening on ports 53, 80, and 443 using a port scanner or by attempting a connection to each port. Fourth, verify that your domain's NS records at your registrar point to your VPS IP. If any of these checks fail, do not proceed to Lab 2 until the issue is resolved.

CHALLENGE QUESTIONS: (1) Why did we open port 53? What would happen if we did not open it, and how would that affect Evilginx2's ability to function? (2) What is the purpose of the UFW firewall rules in the context of Evilginx2, and which port is most critical for the phishing proxy to intercept traffic? (3) Why must Evilginx2 be run as root, and what security implications does this have for your VPS?

COMMON MISTAKES: (1) Forgetting to open DNS port 53 - Evilginx2 must act as an authoritative DNS server to resolve phishing subdomains, and without port 53 open, the Let's Encrypt certificate challenge will fail and no phishing pages will load. (2) Not running Evilginx2 as root - binding to port 53 (a privileged port below 1024) requires root privileges; running as a normal user will result in a "permission denied" error. (3) Not setting the correct public IP in the config command - if you set the internal/private IP instead of the public IP, DNS resolution will point to the wrong address and victims will never reach your phishing page. (4) Forgetting to update NS records at your domain registrar - without this step, your domain will never resolve to your VPS.

Lab 2: Phishlet Configuration and Lure Creation

LEARNING OBJECTIVE: Students will be able to configure a phishlet, create lures, and understand the relationship between phishing domains and real services. By completing this lab, you will know how to enable a phishlet, assign a hostname, generate a lure URL, and customize lure parameters for a realistic phishing simulation.

SCENARIO: Your team lead is satisfied with the test environment from Lab 1. Now they want you to configure a phishlet for a simulated O365 phishing test against CyberSec Corp's test tenant (cybersec-test.onmicrosoft.com). You need to create and customize a lure that looks convincing enough for the upcoming assessment, while ensuring all traffic routes through your Evilginx2 proxy correctly.

PREREQUISITES CHECKLIST:

Before starting this lab, verify: (1) Lab 1 is completed successfully and Evilginx2 is running, (2) Your domain's NS records are pointing to your VPS (verify with "dig NS your-lab-domain.com"), (3) You have a test account on the service you are targeting (e.g., a Microsoft 365 test tenant), (4) You have waited for DNS propagation (can take minutes to hours depending on your registrar).

STEP-BY-STEP INSTRUCTIONS:

Step 1: Enable the O365 phishlet and configure the hostname. The hostname you set will be the subdomain of your lab domain that the phishing page is served from. Choose something that looks plausible, such as "login" or "secure" or "auth".

```
# Inside Evilginx2 CLI:
phishlet enable o365
phishlet hostname o365 login.your-lab-domain.com
```

Step 2: Verify DNS resolution. On your LOCAL machine (not the VPS), use dig or nslookup to confirm that the phishing subdomain resolves to your VPS IP address. If it does not resolve yet, wait for DNS propagation. You can also check that Evilginx2 has obtained a TLS certificate by attempting to visit the URL in a browser and checking for the HTTPS padlock.

```
# On your LOCAL machine (not VPS):
dig login.your-lab-domain.com
# Expected: should resolve to YOUR_VPS_IP
```

Step 3: Create and customize a lure. The lure is the actual phishing link that would be sent to a target. Set a redirect URL that the victim will be sent to after authentication (this should be the real service URL so the victim sees their inbox and nothing seems amiss). Customize the path to something that looks legitimate rather than using the default random path.

```
# Create and customize a lure
lures create o365
lures edit 0 redirect_url https://outlook.office.com
lures edit 0 path /security-update-2026
lures get-url 0
```

VERIFICATION STEPS:

Confirm the following: (1) The lures get-url 0 command produces a valid URL in the format "https://login.your-lab-domain.com/security-update-2026?id=XXXXXX". (2) DNS resolution shows the phishing domain pointing to your VPS IP. (3) Visiting the lure URL in a browser shows the proxied O365 login page with a valid HTTPS certificate. If you see a certificate error, Evilginx2 may not have completed the ACME challenge yet; check the Evilginx2 console for certificate request status.

CHALLENGE QUESTIONS: (1) Why does the redirect_url matter for operational security? What would happen if you set it to a blank page or a 404 error instead of the real service? (2) What would happen if you set the path to just "/" instead of "/security-update-2026"? How does the path choice affect both the credibility of the lure and the ability to run multiple lures simultaneously? (3) How could an attacker use different paths for different targets, and why might this be useful in a real assessment? (4) What happens if you try to create a lure before enabling the phishlet?

COMMON MISTAKES: (1) Not waiting for DNS propagation - this is the number one source of frustration. DNS changes can take anywhere from minutes to 48 hours to propagate globally. Use "dig" to check before assuming the configuration is broken. (2) Using a redirect_url that does not exist - if the redirect URL returns a 404, the victim will suspect something is wrong after logging in. Always use the real service URL as the redirect. (3) Forgetting to enable the phishlet before creating lures - the "lures create" command will fail or produce an unusable URL if the phishlet is not enabled first. (4) Using a hostname that conflicts with an existing DNS record - make sure the subdomain you choose is not already in use.

Lab 3: Full Attack Simulation - Capture and Analyze

LEARNING OBJECTIVE: Students will execute a complete AiTM phishing simulation, capture session tokens, and analyze the captured data to understand what an attacker gains. By completing this lab, you will know how to monitor captured sessions, export captured cookies, and distinguish between authentication tokens and preference cookies in the captured data.

SCENARIO: It is go time. Using your own test account on your own test tenant, you will simulate a complete attack: click the lure, authenticate through the proxy, capture the session, and analyze what was intercepted. This is the most critical lab in the series because it demonstrates the full attack chain

and shows exactly what data an attacker can extract from a successful AiTM phishing attack.

PREREQUISITES CHECKLIST:

Before starting: (1) Labs 1 and 2 are completed successfully, (2) Your lure URL is active and accessible, (3) You have a test account with known credentials on the target service, (4) You have a separate browser profile with NO saved passwords (incognito mode is ideal) - this ensures no password manager auto-fill interferes with the simulation, (5) You understand that you must ONLY use your own test account, never a real person's account.

STEP-BY-STEP INSTRUCTIONS:

Step 1: Verify your lure is active. In the Evilginx2 CLI, get the lure URL and confirm it is accessible.

```
# Step 1: Verify lure is active
lures get-url 0
```

Step 2: Open the lure URL in a SEPARATE browser. Use a browser profile with NO saved passwords. Incognito mode is ideal. This simulates a victim clicking a link from an email. Use YOUR test account credentials only! Never use someone else's credentials, even for testing. The phishing page should display the real O365 login form, proxied through your Evilginx2 server.

Step 3: After authenticating, check for captured sessions. Switch back to the Evilginx2 CLI and list the captured sessions.

```
# Step 3: Check for captured sessions
sessions
```

Step 4: Analyze the captured data in detail. Select a specific session to view its full contents including username, captured cookies, tokens, and timestamps.

```
# Step 4: Analyze captured session
sessions 0
# This shows: username, captured cookies, tokens, timestamps
```

Step 5: Export captured cookies for analysis. Export the session data to a JSON file that can be examined in detail.

```
# Step 5: Export captured cookies
sessions export 0 /tmp/captured_session.json
```

Step 6: Examine the session file. Use Python's JSON formatter to pretty-print the exported session data and examine each captured cookie.

```
# Step 6: Examine session data
cat /tmp/captured_session.json | python3 -m json.tool
```

Now let us examine what each captured cookie means. In a typical O365 session capture, you will find several types of cookies. Authentication cookies (such as "ESTSAUTH", "ESTSAUTHPERSISTENT", and "ESTSAUTHLIGHT") are the most valuable to an attacker because they prove the user has

successfully authenticated. The ESTSAUTHPERSISTENT cookie is particularly dangerous because it is a persistent session token that remains valid even after the browser is closed, potentially for days or weeks. Session tokens (such as " SignInStateCookie " or "brcap") contain the actual session identifier that the server uses to look up the authenticated session. Preference cookies (such as "MUID", "_Ues", or "OH.DCAfl") track user preferences, analytics, and feature flags; these are useless for session replay but may reveal information about the user's environment. The critical takeaway is that an attacker only needs the authentication cookies to replay the session; the other cookies are noise.

VERIFICATION STEPS:

Confirm the following: (1) The sessions command shows at least one captured session with your test account username and the capture type "cookies + creds". (2) The JSON export file exists and contains session tokens, not just preference cookies. (3) You can identify which cookies are authentication tokens versus preference cookies by examining the cookie names and values. (4) The timestamp on the captured session matches the time you performed the authentication.

CHALLENGE QUESTIONS: (1) Looking at the captured session data, which specific cookie would an attacker need to replay the session? What makes this cookie different from the others? (2) How long is this cookie valid, and how could you determine the expiration time from the captured data or from documentation? (3) What is the difference between the captured credentials (username and password) and the captured session token? Why does having the session token make the credentials less valuable to the attacker? (4) Why does changing the password NOT invalidate the session? What would the account owner need to do to revoke the stolen session? (5) If the session token expires in 24 hours, does that mean the attacker loses access after 24 hours, or are there mechanisms that could extend access?

COMMON MISTAKES: (1) Using a real account instead of a test account - NEVER do this, even for "quick testing." This is both illegal and unethical. Always use accounts you own on tenants you control. (2) Forgetting to check sessions immediately after authentication - some tokens expire quickly, and if you wait too long, the session data may be stale or incomplete. (3) Not understanding the difference between first-party and third-party cookies in the capture - first-party cookies set by the authentication domain are the ones needed for replay; third-party cookies from analytics or CDNs are not useful for session replay. (4) Attempting to use the captured password instead of the session token for replay - the password alone will trigger 2FA on the real site, defeating the purpose of capturing the session token.

Lab 4: Defense Verification - Testing Your Organization's Defenses

LEARNING OBJECTIVE: Students will verify whether common defensive measures successfully block or detect the AiTM attack, and learn how to validate security controls. By completing this lab, you will be able to test password manager behavior, browser security indicators, session replay from different IPs, and the effectiveness of FIDO2 authentication against AiTM attacks.

SCENARIO: Now you switch hats from red team to blue team. Your team lead wants you to verify that CyberSec Corp's defenses would catch this attack. You will test four specific defenses: (1) Does the password manager auto-fill on the phishing domain? (2) Does the browser show any warnings that could alert the user? (3) Can you replay the session from a different IP? (4) What happens if the target account

has FIDO2 enabled? This lab is about validating defenses, not breaking them.

PREREQUISITES CHECKLIST:

Before starting: (1) Labs 1-3 are completed successfully, (2) You have a captured session from Lab 3, (3) You have a password manager installed and configured with your test account credentials, (4) You have access to a second machine or VPN with a different IP address for the session replay test, (5) (Optional) You have a FIDO2 security key like a YubiKey.

STEP-BY-STEP INSTRUCTIONS:

Test 1: Password Manager Test. Configure your password manager (Bitwarden, 1Password, etc.) with your test account credentials. Then visit the lure URL from Lab 2. OBSERVE: Does the password manager auto-fill? EXPECTED: It should NOT auto-fill because the domain is different from the real service. The password manager matches URLs against its stored entries, and login.your-lab-domain.com does not match login.microsoftonline.com. If it DOES auto-fill, this is a security failure that should be reported to the password manager vendor.

```
# Test 1: Password Manager Behavior
# 1. Configure password manager with test account credentials
# 2. Visit the lure URL
# 3. OBSERVE: Does the password manager auto-fill?
# 4. EXPECTED: It should NOT auto-fill (different domain)
# 5. If it DOES auto-fill, this is a security failure
```

Test 2: Browser Security Indicators. Visit the lure URL in Chrome, Firefox, and Edge. OBSERVE: What does the address bar show? Check: Is the domain visible? Is the HTTPS lock present? Note that the lock WILL be present because Evilginx2 obtains a legitimate Let's Encrypt certificate, but the DOMAIN will be wrong. This is a critical observation: the HTTPS padlock misleads users into trusting the site, which is why user education must focus on verifying the domain name, not just the presence of the padlock.

```
# Test 2: Browser Security Indicators
# 1. Visit lure URL in Chrome, Firefox, and Edge
# 2. OBSERVE: What does the address bar show?
# 3. Check: Is the domain visible? Is the HTTPS lock present?
# 4. Note: Lock WILL be present (Let us Encrypt cert)
# but the DOMAIN will be wrong
```

Test 3: Session Replay from Different IP. Capture a session from Lab 3. Export the cookies. From a DIFFERENT machine or IP address (e.g., through a VPN or from your local machine instead of the VPS), attempt to inject the cookies using EditThisCookie or a similar browser extension. Navigate to the real service and see if the session is valid. OBSERVE: Does the session work from a different IP? If Conditional Access policies requiring compliant devices or specific IP ranges are enabled, the session should be blocked.

```
# Test 3: Session Replay from Different IP
# 1. Capture a session (from Lab 3)
# 2. Export cookies
# 3. From a DIFFERENT machine/IP, inject cookies
# using EditThisCookie or similar browser extension
# 4. OBSERVE: Does the session work from a different IP?
# 5. If Conditional Access is enabled, it should be blocked
```

Test 4: FIDO2 Test. Enable FIDO2/WebAuthn on your test account. Visit the lure URL and attempt to authenticate. OBSERVE: Does the FIDO2 key authenticate? EXPECTED: FIDO2 should FAIL because of the domain mismatch. When the browser prompts for FIDO2 authentication on the phishing domain, the security key checks the Relying Party ID (the domain) and finds that it does not match the domain where the credential was registered. The key refuses to sign the challenge, and authentication fails. This proves FIDO2 is the effective defense against AiTM phishing attacks.

```
# Test 4: FIDO2 Test
# 1. Enable FIDO2/WebAuthn on your test account
# 2. Visit the lure URL and attempt to authenticate
# 3. OBSERVE: Does the FIDO2 key authenticate?
# 4. EXPECTED: FIDO2 should FAIL (domain mismatch)
# 5. This proves FIDO2 is the effective defense
```

VERIFICATION STEPS:

For each test, document your findings: (1) Test 1: Record whether the password manager auto-filled. If it did not, the defense is working correctly. If it did, document the password manager and version for a findings report. (2) Test 2: Screenshot the address bar in each browser. Document whether the HTTPS padlock is present and whether the domain is clearly visible. (3) Test 3: Record whether the session was replayable from a different IP. If blocked, document what error message or conditional access prompt was displayed. (4) Test 4: Record the exact error message when FIDO2 authentication failed on the phishing domain. This proves the domain-binding property of FIDO2.

CHALLENGE QUESTIONS: (1) If the password manager auto-fills on the phishing domain, what does this mean for the organization's security posture? How would you rate the severity of this finding in a penetration test report? (2) Why does the HTTPS padlock appear on the phishing site, and how does this mislead users? How would you explain this to a non-technical executive who believes "the padlock means the site is safe"? (3) How would you explain to a non-technical executive why SMS-based 2FA is insufficient against this attack? Use an analogy that makes the concept accessible. (4) If Conditional Access blocks the session replay from a different IP, does that mean the organization is fully protected? What other attack vectors might still work? (5) Why is FIDO2 effective while SMS 2FA is not, given that both are "two-factor" methods?

COMMON MISTAKES: (1) Confusing "the HTTPS padlock is present" with "the site is legitimate" - this is the most common user misconception. The padlock only proves the connection is encrypted, not that the domain belongs to the expected organization. (2) Assuming that because one browser shows a warning, all browsers will - different browsers have different phishing detection capabilities and UI treatments. Test in all browsers your organization uses. (3) Forgetting to test from a genuinely different IP address in Test 3 - if you test from the same network, the IP-based conditional access may not trigger. (4) Not enabling FIDO2 before

testing - some students skip the FIDO2 test because they do not have a security key, but this is the most important test. Even if you do not have a physical key, understand and document the expected behavior. (5) Not documenting findings - in a real assessment, every test result must be documented with evidence. Take screenshots, record error messages, and note exact timestamps.

Table 6: Skills Assessment Rubric

Skill	Beginner	Intermediate	Advanced
Environment Setup	Can install Evilginx2 with guidance	Can configure independently	Can troubleshoot DNS/TLS issues
Phishlet Config	Can enable built-in phishlets	Can customize lure parameters	Can create custom phishlets
Attack Execution	Can capture a session with guidance	Can run full simulation independently	Can adapt to unexpected errors
Defense Testing	Can verify FIDO2 works	Can test multiple defensive layers	Can write professional findings report

Lab Completion Certificate

Upon completing all four labs, you should be able to perform the following tasks independently, without referring to the step-by-step instructions in this guide:

1. Provision and harden a VPS for running Evilginx2, including correct firewall rules and DNS configuration. 2. Install, compile, and configure Evilginx2 from source, including setting the base domain and public IP address. 3. Enable and configure built-in phishlets for common services, including setting hostnames and verifying TLS certificate issuance. 4. Create and customize lures with appropriate redirect URLs and paths for realistic phishing simulations. 5. Execute a complete AiTM phishing simulation from lure click to session capture, using only accounts and infrastructure you own. 6. Analyze captured session data to identify authentication tokens versus preference cookies, and explain the significance of each. 7. Export and examine captured session data in JSON format for further analysis or reporting. 8. Test password manager auto-fill behavior on phishing domains and interpret the results. 9. Evaluate browser security indicators (HTTPS padlock, domain display) across multiple browsers. 10. Verify the effectiveness of Conditional Access policies by attempting session replay from different IP addresses. 11. Demonstrate why FIDO2/WebAuthn is the gold standard defense by showing authentication failure on phishing domains. 12. Write a professional findings report documenting defensive test results with evidence, severity ratings, and remediation recommendations.

If you can perform all twelve of these tasks independently, you have successfully completed the practice lab and have demonstrated both offensive and defensive competencies in AiTM phishing assessment. Remember: the skills you have practiced are powerful tools that must only be used with explicit authorization. The ethical principles discussed in Chapter 9 are not optional; they are the foundation of a legitimate cybersecurity career. Carry forward the knowledge you have gained, use it to protect organizations and individuals, and never stop learning.